

Excel – segédlet

Emelt digitális kultúra · táblázatkezelés · Excel 2021 · magyar függvénynevek, pontosvessző elválasztóval

Kezdő ellenőrzőlista

- **Import:** Adatok → Szövegből/CSV. Állítsd be a **tagolót** (;), a **tizedesjelet** (vessző/pont) és a **kódolást** (UTF-8 vagy 1250). Ha számok szöveggént jönnek be, minden képlet hibás lesz.
- A **forrásadatot ne rendezd át** és ne írd felül. Rendezett eredményt külön munkalapra / külön területre tedd, ha a feladat másként nem kéri.
- Képletet **töltsd végig**, és nézd meg az **utolsó sort** is. Fix érték helyett mindig képlet. Rögzítés: **\$A\$1** (F4).
- Segédoszlop szabad, ha a feladat nem tiltja. Két rövid képlet jobb, mint egy átláthatatlan hosszú.
- Formátum: a feladat előírja (tizedesek száma, ezres tagolás, pénznem, dátumformátum). Ez külön pont.
- **Diagram:** típus, cím, tengelycímek, jelmagyarázat, adatfeliratok, tengelyskála, elhelyezés a megadott helyre.
- Mentés a megadott névvel és formátummal (.xlsx), gyakran. A munkalap nevét is a feladat szerint add meg.

Függvények (magyar = angol)

Magyar	Angol	Mire jó
HA / ÉS / VAGY / NEM	IF / AND / OR / NOT	Feltétel, logika
ÁTVÁLT	SWITCH	Egy érték több lehetséges értékkel párosítva
HAHIBA	IFERROR	Hiba lecserélése
SZUM, ÁTLAG, MIN, MAX	SUM, AVERAGE, MIN, MAX	Alap összesítések
DARAB / DARAB2 / DARABÜRES	COUNT / COUNTA / COUNTBLANK	Számok / nem üres / üres
DARABTELI / DARABHATÓBB	COUNTIF / COUNTIFS	Feltételes darabszám
SZUMHA / SZUMHATÓBB	SUMIF / SUMIFS	Feltételes összeg
ÁTLAGHA / ÁTLAGHATÓBB	AVERAGEIF / AVERAGEIFS	Feltételes átlag
MAXHA / MINHA	MAXIFS / MINIFS	Feltételes maximum / minimum
SZORZATÖSSZEG	SUMPRODUCT	Összetett feltételek, súlyozás
XKERES	XLOOKUP	Keresés bármely irányba, ez a fő keresőfüggvény
FKERES / VKERES / INDEX + HOL.VAN	VLOOKUP / HLOOKUP / INDEX + MATCH	Régebbi keresések (FKERES 4. arg.: HAMIS = pontos)
SZŰRŐ / EGYEDI	FILTER / UNIQUE	Dinamikus tömb: egy cellába írod, a többibe kiömlik
SORBA.RENDEZ / RENDEZÉS.ALAP.SZERINT	SORT / SORTBY	Rendezés képlettel (2. arg.: oszlop, 3. arg.: 1 növ., -1 csökk.)
BAL / JOBB / KÖZÉP / HOSSZ	LEFT / RIGHT / MID / LEN	Szövegrészek
SZÖVEG.KERES / SZÖVEG.TALÁL	SEARCH / FIND	Pozíció (a KERES kis/nagybetű-független)
CSERE / HELYETTE / KIMETSZ	REPLACE / SUBSTITUTE / TRIM	Szöveg átalakítás
ÖSSZEFÜZ / FÜZ / SZÖVEGÖSSZEFÜZÉS	CONCATENATE / CONCAT / TEXTJOIN	Összefűzés (a TEXTJOIN elválasztóval, tartományból is)
SZÖVEG / ÉRTÉK	TEXT / VALUE	Szám↔szöveg
KEREKÍTÉS / KEREK.FEL / KEREK.LE	ROUND / ROUNDUP / ROUNDDOWN	Kerekítés
EGÉSZ / MARADÉK	INT / MOD	Egész rész / osztási maradék
ÉV / HÓNAP / NAP / HÉT.NAPJA	YEAR / MONTH / DAY / WEEKDAY	Dátum részei (HÉT.NAPJA 2. arg.: 2 = hétfő az 1.)
DÁTUM / MA	DATE / TODAY	Dátum képzés; két dátum különbsége sima kivonás
NAGY / KICSI / RANG	LARGE / SMALL / RANK	k-adik legnagyobb / sorrend

Mintaképletek

Cél	Képlet
Feltétel	=HA(B2>=50;"megfelelt";"nem felelt meg")
Több feltétel	=HA(ÉS(B2>=50;C2="igen");1;0)
Pontos keresés	=XKERES(A2;\$H\$2:\$H\$20;\$I\$2:\$I\$20;"nincs")
Sávós besorolás (nem kell rendezés)	=XKERES(B2;\$K\$2:\$K\$6;\$L\$2:\$L\$6;-1) (-1: pontos, vagy a következő kisebb)
Max. értékhez tartozó név	=XKERES(MAX(D2:D100);D2:D100;A2:A100)
Feltételes összeg	=SZUMHATÓBB(\$D\$2:\$D\$100;\$B\$2:\$B\$100;F2;\$C\$2:\$C\$100;">"&G2)
Feltételes maximum	=MAXHA(\$D\$2:\$D\$100;\$B\$2:\$B\$100;F2)
Darabszám, szövegminta	=DARABTELI(\$B\$2:\$B\$100;"A*")
Szorzat-összeg feltétellel	=SZORZATÖSSZEG((\$B\$2:\$B\$100="x")*\$D\$2:\$D\$100)
Szűrt lista (kiömlik)	=SZŰRŐ(A2:D100;C2:C100>50;"nincs")
Egyedi, rendezett lista	=SORBA.RENDEZ(EGYEDI(B2:B100)) · darabszám: =DARAB2(EGYEDI(B2:B100))
Rendezés oszlop szerint	=SORBA.RENDEZ(A2:D100;4;-1) (4. oszlop szerint csökkenő)
Szöveg első szava	=BAL(A2;SZÖVEG.KERES(" ";A2)-1)
Szám formázott szöveggé	=SZÖVEG(D2;"0,00")&" Ft"
Összefűzés elválasztóval	=SZÖVEGÖSSZEFÜZÉS(" ";IGAZ;A2:A10)

Gyakori hibák és trükkök

- **#HIÁNYZIK:** nincs találat vagy a keresett érték szöveg/szám típusa eltér (XKERES-nél a 4. argumentummal kezelhető). **#ÉRTÉK!**: szöveg a számítási oszlopban. **#HIV!** / **#NÉV?**: rossz hivatkozás vagy függvénynév.
- **XKERES:** =XKERES(keresett; keresési_tömb; visszaadott_tömb; [ha_nincs]; [illesztés]; [keresési_mód]). Illesztés: **0** pontos (alap), **-1** pontos vagy következő kisebb, **1** pontos vagy következő nagyobb. Üres argumentumot kihagyni ;;-vel lehet.
- **Dinamikus tömb:** elég az első cellába írni, a többi kiömlik. Ha a kiömlési tartományban van adat, kiömlési hibát (**#SPILL!**) kapsz. A teljes kiömlött tartományra **A2#** hivatkozik, és ezt fogadják a diagramok és más függvények.
- Számformátum: egyedi **0,00** · **#** **##0** · **0,0%** · **#** **##0** **"Ft"** · dátum: **éééé.hh.nn**. Cellaformátum ≠ érték: a formátum csak megjelenítés.
- Feltételes formázás: **Kezdőlap** → **Feltételes formázás** → **Új szabály** → **Képlet**. A képlet az első cellára hivatkozzon, a sorrögzítést figyelj (\$A2).
- Kimutatás: Beszúrás → Kimutatás. Az érték mezőnél állítsd az összegzés típusát (összeg/darab/átlag) és a számformátumot. Gyorsbillentyűk: **Ctrl+Shift+L** szűrő, **F4** \$ váltás, **Alt+=** automatikus összeg.

SQL – segédlet

MySQL / MariaDB · phpMyAdmin vagy Workbench · a MySQL 8-as újdonságokat külön jelöltem

Munkamenet

- **Minden lekérdezést mentsd el** egy szövegfájlba a feladatszámmal. Ellenőrizd az **oszlopsorrendet**, az **álneveket** (AS), a **rendezést** és a **sorok számát** a minta alapján.
- Először egy egyszerű SELECT, utána add hozzá a JOIN-t, a WHERE-t, a GROUP BY-t. Mindig nézd meg a köztes eredményt.
- Tájékozódás: **SHOW TABLES**; · **DESCRIBE** *tabla*; · **SHOW CREATE TABLE** *tabla*; · **USE** *adatbázis*; Script futtatása: Import fül vagy **SOURCE** *fajl.sql*; Kódolás: utf8mb4.

Alapszerkezet (és a végrehajtási sorrend)

```
SELECT t.csoport, COUNT(*) AS db, ROUND(AVG(t.ertek), 2) AS atlag
FROM      tabla1 t
INNER JOIN tabla2 u ON t.id = u.tabla1_id
WHERE     t.datum >= '2020-01-01' AND t.nev LIKE 'A%'
GROUP BY t.csoport
HAVING    COUNT(*) > 3
ORDER BY db DESC, t.csoport ASC
LIMIT 10;
-- végrehajtás: FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

Gyakori minták

Cél	Minta
Legnagyobb értékű sor	SELECT * FROM t WHERE ertek = (SELECT MAX(ertek) FROM t);
Top 3 / lapozás	SELECT * FROM t ORDER BY ertek DESC LIMIT 3; LIMIT 5, 10 (kihagy 5-öt, kér 10-et)
Átlag feletti	SELECT * FROM t WHERE ertek > (SELECT AVG(ertek) FROM t);
Különböző értékek száma	SELECT COUNT(DISTINCT csoport) FROM t;
Amihez nincs kapcsolódó sor	SELECT a.* FROM a LEFT JOIN b ON a.id = b.a_id WHERE b.a_id IS NULL;
Feltételes darabszám	SUM(CASE WHEN ertek > 5 THEN 1 ELSE 0 END) AS sok vagy röviden: SUM(ertek > 5)
Csoportonkénti legnagyobb sor	SELECT t.* FROM t JOIN (SELECT csoport, MAX(ertek) AS m FROM t GROUP BY csoport) x ON t.csoport = x.csoport AND t.ertek = x.m;
Csoport elemei egy cellában	SELECT csoport, GROUP_CONCAT(nev ORDER BY nev SEPARATOR ', ') FROM t GROUP BY csoport;
Rangsor csoportonként (8.0+)	ROW_NUMBER() OVER (PARTITION BY csoport ORDER BY ertek DESC) AS hely
Szűrés	WHERE csoport IN ('A','B') · ertek BETWEEN 10 AND 20 · nev IS NOT NULL
Szövegminta	LIKE 'A%' (A-val kezdődik) · '%ab%' (tartalmaz) · '_a%' (2. betű a)
Két lekérdezés egyesítése	SELECT nev FROM a UNION SELECT nev FROM b; (UNION ALL: duplikátumokkal)
Új sor / több sor	INSERT INTO t (a, b) VALUES (1, 'x'), (2, 'y');
Módosítás	UPDATE t SET ertek = ertek + 1 WHERE id = 5;
Módosítás másik tábla alapján	UPDATE a JOIN b ON a.id = b.a_id SET a.ar = b.ar WHERE b.aktiv = 1;
Törlés	DELETE FROM t WHERE id = 5; -- WHERE nélkül mindent töröl!
Tábla létrehozása	CREATE TABLE t (id INT AUTO_INCREMENT PRIMARY KEY, nev VARCHAR(50) NOT NULL, datum DATE, a_id INT, FOREIGN KEY (a_id) REFERENCES a(id));
Tábla módosítása	ALTER TABLE t ADD COLUMN megj VARCHAR(100); · MODIFY nev VARCHAR(80); · DROP COLUMN megj;
Al-lekérdezés a FROM-ban	SELECT AVG(db) FROM (SELECT csoport, COUNT(*) AS db FROM t GROUP BY csoport) AS x; (alias kötelező)

MySQL függvények

	Függvények
Dátum	YEAR(d) MONTH(d) DAY(d) DAYOFWEEK(d) (1 = vasárnap) WEEKDAY(d) (0 = hétfő) DATEDIFF(d1, d2) DATE_ADD(d, INTERVAL 7 DAY) TIMESTAMPDIF(YEAR, szul, CURDATE()) DATE_FORMAT(d, '%Y.%m.%d') CURDATE() NOW() HOUR(t) MINUTE(t) TIMEDIFF(t1, t2)
Szöveg	CONCAT(a, b) CONCAT_WS(',', a, b) SUBSTRING(s, p, h) LEFT(s, n) RIGHT(s, n) CHAR_LENGTH(s) UPPER LOWER TRIM REPLACE(s, a, b) LOCATE('x', s) LPAD(s, 5, '0')
Szám	ROUND(x, n) TRUNCATE(x, n) FLOOR CEILING ABS MOD(a, b) vagy a % b a DIV b (egész osztás) POWER(x, n) SQRT(x)
NULL, feltétel	IFNULL(x, 0) COALESCE(x, y, 0) NULLIF(a, b) IF(feltétel, igaz, hamis) CASE WHEN ... THEN ... ELSE ... END
Összesítők	COUNT SUM AVG MIN MAX GROUP_CONCAT
MySQL 8.0+	ROW_NUMBER() RANK() DENSE_RANK() OVER (PARTITION BY ... ORDER BY ...) · WITH nev AS (SELECT ...) SELECT ... FROM nev;

Gyakori buktatók

- A **WHERE-ben nem lehet aggregátum** (COUNT, SUM, AVG...), oda a HAVING való. MySQL-ben az álnevet a GROUP BY, a HAVING és az ORDER BY használhatja, a WHERE nem.
- **ONLY_FULL_GROUP_BY** (alapértelmezett): minden nem aggregált SELECT-oszlopnak szerepelnie kell a GROUP BY-ban, különben 1055-ös hibát kapsz.
- **COUNT(*)** minden sort számol, **COUNT(oszlop)** a NULL-t kihagyja. NULL-ra **IS NULL**, soha nem = NULL. Rendezésnél a NULL növekvőnél előre kerül.
- A / MySQL-ben tizedest ad (5/2 = 2.5), az egész osztás a **DIV**. A szövegek összehasonlítása és a LIKE alapból **kis/nagybetű-független**.
- JOIN után ellenőrizd, nem szaporodtak-e meg a sorok (hiányzó feltétel, rossz kapcsoló oszlop). Szükség esetén DISTINCT.
- Szöveg és dátum **aposztrófban**: '2020-01-01'. Backtick (`) csak kulcsszó vagy szóközös név előtt. Megjegyzés: -- (a két kötőjel után szóköz kell!), #, /* ... */.
- **UPDATE / DELETE WHERE nélkül mindent érint**. Ha a Workbench 1175-ös (safe update) hibát ad, a kulcsra (id) szűrj, ne kapcsold ki a védelmet.
- Pont a megadott **oszlopnevekkel** és **sorrenddel** kéri az eredményt. Ha a mintában 12 sor van és te 13-at kapsz, valahol szűrés vagy JOIN hibázik.

C++ – segédlet

Code::Blocks · fájlbeolvasás, programozási tételek, STL, formázás, hibakeresés

Munkamenet és Code::Blocks

- **Készíts sablont**, és írd le fejből gyakorláskor: include-ok, struct, beolvasás, a feladatok sorban. Ez az első 5–10 perc.
- A **feladatokat sorban**, külön függvényben vagy blokkban old meg, kommenttel (// 3. feladat). Ami nem fordul, tedd megjegyzésbe, hogy a többi lefusszon.
- Hasonlítsd össze a **kimenetet a mintával** betű szerint: szóközök, kettőspont, sortörés, tizedesek.
- **Billentyűk**: Build = Ctrl+F9, Build and run = F9, Debug = F8. A fordító standardját itt állítod: Settings → Compiler → *Have g++ follow the C++17 ISO C++ language standard*.
- A **forrásfájlt (forras.txt)** abba a mappába tedd, ahonnan a program fut (általában a projekt mappája), különben nem találja. Ellenőrizd: **if (!be) { cout << "Nincs fájl"; return 1; }**. A kimeneti fájl is oda kerül.
- Nézd a **Build messages** ablakot is: a figyelmeztetések (uninitialized, signed/unsigned) gyakran a hiba okát mutatják.

Sablon: beolvasás struct-ba

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <algorithm>
#include <iomanip>
#include <cmath>
#include <map>
#include <set>
#include <numeric>
using namespace std;

struct Adat { string nev; int ertekek; double ar; };

int main() {
    ifstream infile("forras.txt");
    if (!infile.is_open()) { cout << "Nincs fajl" << endl; return 1; }
    string sor;
    getline(infile, sor); // fejléc kihagyása (ha van)
    vector<Adat> v;
    while (getline(infile, sor)) {
        if (!sor.empty() && sor.back() == '\r') sor.pop_back(); // Windows sorvég
        if (sor.empty()) continue;
        stringstream ss(sor);
        string nev, e, a;
        getline(ss, nev, ';'); getline(ss, e, ';'); getline(ss, a, ';');
        replace(a.begin(), a.end(), ',', '.'); // tizedesvessző -> pont
        v.push_back({nev, stoi(e), stod(a)});
    }
    be.close();

    cout << "1. feladat: " << v.size() << endl;
    return 0;
}
```

Beolvasási változatok

```
// 1) Első sor a darabszám, utána szóközzel tagolt adatok (a név egy szó legyen)
int n; be >> n; vector<Adat> v(n);
for (int i = 0; i < n; i++) be >> v[i].nev >> v[i].ertekek;

// 2) Általános felbontó függvény (bármilyen elválasztóra)
vector<string> split(const string& s, char d) {
    vector<string> r; string t; stringstream ss(s);
    while (getline(ss, t, d)) r.push_back(t);
    return r;
}
auto m = split(sor, ';'); // m[0], m[1], ... stoi(m[1]), stod(m[2])

// 3) Szavakra bontás szóköz mentén
stringstream ss(sor); string szo; while (ss >> szo) { /* szo feldolgozása */ }

// 4) Mátrix: n sor, k oszlop
vector<vector<int>> mx(n, vector<int>(k));
for (int i = 0; i < n; i++) for (int j = 0; j < k; j++) be >> mx[i][j];

// 5) Billentyűzetről
int x; cout << "Kerek egy szamot: "; cin >> x;
cin.ignore(); getline(cin, szoveg); // >> után ignore, különben a getline üres sort olvas
```

Programozási tételek

```
// Összegezés, átlag
long long ossz = 0; for (auto& x : v) ossz += x.ertek; double atlag = (double)ossz / v.size();
// Megszámolás
int db = 0; for (auto& x : v) if (x.ertek > 10) db++;
// Maximum-kiválasztás (index): > az ELSŐ legnagyobbát adja, >= az UTOLSÓT
int mi = 0; for (int i = 1; i < (int)v.size(); i++) if (v[i].ertek > v[mi].ertek) mi = i;
// Az összes maximális elem
int mx = max_element(v.begin(), v.end(), [](auto& a, auto& b){ return a.ertek < b.ertek; })->ertek;
for (auto& x : v) if (x.ertek == mx) cout << x.nev << endl;
// Eldöntés (van-e ilyen?)
bool van = false; for (auto& x : v) if (x.nev == "x") { van = true; break; }
// Keresés (index vagy -1)
int hely = -1; for (int i = 0; i < (int)v.size() && hely == -1; i++) if (v[i].nev == "x") hely = i;
// Kiválogatás, szétválogatás
vector<Adat> r; for (auto& x : v) if (x.ertek > 10) r.push_back(x);
vector<Adat> kicsi, nagy; for (auto& x : v) (x.ertek < 10 ? kicsi : nagy).push_back(x);
// Rendezés több szempont szerint (stable_sort: az egyenlők eredeti sorrendje megmarad)
sort(v.begin(), v.end(), [](const Adat& a, const Adat& b) {
    if (a.ertek != b.ertek) return a.ertek > b.ertek; // 1. érték csökkenő
    return a.nev < b.nev; // 2. név növekvő
});
// Halmazműveletek rendezett set-ekkel
set<int> A, B, met; set_intersection(A.begin(), A.end(), B.begin(), B.end(), inserter(met, met.begin()));
```

Segédfüggvények (számelmélet, idő, szöveg)

```
bool prim(int n) {
    if (n < 2) return false;
    for (int i = 2; (long long)i * i <= n; i++) if (n % i == 0) return false;
    return true;
}
int szamjegyOsszeg(int n) { int s = 0; for (n = abs(n); n > 0; n /= 10) s += n % 10; return s; }
bool szokoev(int ev) { return (ev % 4 == 0 && ev % 100 != 0) || ev % 400 == 0; }
int ltko(int a, int b) { return b == 0 ? a : ltko(b, a % b); } // C++17: gcd(a, b) (<numeric>)
bool palindrom(string s) { string r = s; reverse(r.begin(), r.end()); return s == r; }
string binaris(int n) { string r; do { r = char('0' + n % 2) + r; n /= 2; } while (n > 0); return r; }
int perc = 135; cout << perc / 60 << ":" << setfill('0') << setw(2) << perc % 60 << endl; // 2:15
cout << setfill(' '); // visszaállítás, különben a többi setw is 0-val tölt
```

STL, vector, map

Cél	Kód
Rendezés	sort(v.begin(), v.end(), [](const Adat& a, const Adat& b){ return a.ertek > b.ertek; });
Megszámolás	int db = count_if(v.begin(), v.end(), [](const Adat& a){ return a.ertek > 10; });
Maximum / minimum	auto it = max_element(v.begin(), v.end(), [](auto& a, auto& b){ return a.ertek < b.ertek; }); it->nev (min_element: ugyanígy)
Keresés	auto it = find_if(v.begin(), v.end(), [](const Adat& a){ return a.nev == "x"; }); it != v.end() (indexe: it - v.begin())
Összeg	int ossz = accumulate(v.begin(), v.end(), 0, [](int s, const Adat& a){ return s + a.ertek; });
Mindegyik / valamelyik	all_of(v.begin(), v.end(), pred) any_of(...) none_of(...)
Gyakoriság (map, kulcs szerint rendezett)	map<string,int> db; for (auto& x : v) db[x.nev]++; for (auto& p : db) cout << p.first << " " << p.second << endl; db.count(k)
Különböző elemek	set<string> egyedi; for (auto& x : v) egyedi.insert(x.nev); egyedi.size()
Duplikátum törlése	sort(v.begin(), v.end()); v.erase(unique(v.begin(), v.end(), v.end()));
vector alapok	push_back pop_back size() empty() front() back() clear() resize(n) v.erase(v.begin() + i) v.insert(v.begin() + i, x)
Fordítás, kitöltés	reverse(v.begin(), v.end()); fill(v.begin(), v.end(), 0); iota(v.begin(), v.end(), 1); swap(a, b); min(a, b); max(a, b)
Bináris keresés (rendezettre)	binary_search(v.begin(), v.end(), x)
2D vector	vector<vector<int>> m(n, vector<int>(k, 0)); m[i][j]

Szöveg és karakter

Cél	Kód
Karakter, rész	s[i] s.front() s.back() s.size() s.substr(p, h) s.find("x") (npos, ha nincs) s.rfind("x")
Módosítás	s += "x"; s.insert(p, "x"); s.erase(p, h); s.replace(p, h, "x"); reverse(s.begin(), s.end());
Átalakítás	to_string(n) stoi(s) stod(s) c - '0' (számjegy értéke) (char)('0' + n)
Kis/nagybetű	for (char& c : s) c = toupper(c); (tolower: kisbetű) #include <cctype>
Karaktervizsgálat	isdigit(c) isalpha(c) isupper(c) islower(c) isspace(c)
'a' és "a"	'a' egy karakter (char), "a" szöveg (string). Összehasonlítás: s == "alma", s < t (ábécésorrend).

Kiírás és fájlba írás

```
cout << fixed << setprecision(2) << 3.14159; // 3.14 (fixed nélkül: az értékes jegyek száma!)
cout << left << setw(15) << x.nev << right << setw(6) << x.ertek << endl; // oszlopos kiírás
cout << (van ? "igen" : "nem");
double kerekített = round(x * 100) / 100; // #include <cmath>
ofstream ki("kimenet.txt"); ki << "1. feladat: " << db << endl; ki.close(); // mint a cout
```

Gyakori buktatók

- **Egész osztás:** $5 / 2 = 2$. Átlaghoz kell egy **(double)** cast: `(double)össz / v.size()`. A **%** negatív számnál negatív maradékot adhat.
- **Inicializálatlan változó:** `int db`; helyett **`int db = 0;`**. Összegnél 0, maximumnál az első elem vagy egy nagyon kicsi érték, minimumnál egy nagyon nagy.
- **Túlcsoordulás:** nagy összegeknél, szorzatnál, faktoriálisnál **`long long`**. A **`v.size()`** előjel nélküli: `i < v.size()` - 1 üres vektornál hibás, ilyenkor `(int)v.size()`.
- **= és ==:** `if (x = 5)` értékadás! Az **`if (...);`** végén lévő pontosvessző üres törzset jelent. A **`char`** és a **`string`** idézőjele különbözik.
- **Off-by-one:** a vektor indexe 0-tól `v.size()-1`-ig megy; `v[v.size()]` hibás. Hibakereséshez használd a **`v.at(i)`** hívást, ami kivételt dob az érvénytelen indexnél.
- **Tizedesek összehasonlítása:** `double`-t ne vess össze `==`-vel, használd az **`abs(a - b) < 1e-9`** alakot.
- **Windows sorvég (\r):** ha a beolvasott utolsó mező furcsán viselkedik, vágd le a sor végéről. **Tizedesvessző** a fájlban: stod előtt cseréld pontra.
- **cin >> után getline** üres sort kap: közé kell a `cin.ignore()`. A **`setfill('0')`** bent marad, amíg vissza nem állítod.
- **Ékezetek:** a konzol és a fájl kódolása eltérhet. Fájlba írásnál a fájlt nézd meg, nem a konzolt.
- **Ne írd fix számot**, ha az az adat méretéből adódik: használd a `v.size()`-t. A megoldás ne egy konkrét fájlra legyen ráhúzva.
- Ha a feladat sorszámozott kimenetet ad (1. feladat: ...), **pontosan így írd ki** a feladatszámot és a szöveget.